

Entrega #4 Mini-lenguaje

Jesús Ismael Ramos Recio

Direcciones virtuales

Se desarrolló en el archivo de la Semántica del mini-lenguaje un sistema de direcciones virtuales donde representan las ubicaciones de los datos en la memoria del compilador. Cada dato recibe una dirección numérica única. Las direcciones se dividieron en rangos de direcciones del 1 al 9999. Los rangos de direcciones están definidos de la siguiente forma:

Tipo de dato	Rango
Constantes	1 a 1999
Variables globales	2000 a 3999
Variables locales	4000 a 5999
Parámetros	6000 a 7999
Temporales	8000 a 9999

Y en código están definidos de esta forma:

```
self.rango_dir_cte = 1 # Constantes
    self.rango_dir_g = 2000 # Var globales
    self.rango_dir_l = 4000 # Var locales
    self.rango_dir_p = 6000 # Parametros
    self.rango_dir_temp = 8000 # var temporales
```

Y estas direcciones se almacenarán en un diccionario respecto a su rango definido:

```
# Dicciones de almacenamiento de direcciones asignadas
    self.dir_cte = {}
    self.dir_g = {}
    self.dir_l = {}
    self.dir_p = {}
    self.dir_temp = {}
```

Los siguientes métodos listados se encargan del manejo de asignar las direcciones a los tipos de datos. Debajo de la definición del método está comentado su propósito.

```
def nueva_cte(self, valor):
```

```

        # Asignar dirección virtual a una constante
        if valor not in self.dir_cte:
            self.cont_cte += 1
            direccion = self.rango_dir_cte + self.cont_cte - 1
            self.dir_cte[valor] = direccion
        return self.dir_cte[valor]

def nueva_var_g(self, nombre):
    # Asignar dirección virtual a una variable global
    if nombre not in self.dir_g:
        self.cont_g += 1
        direccion = self.rango_dir_g + self.cont_g - 1
        self.dir_g[nombre] = direccion
    return self.dir_g[nombre]

def nueva_var_l(self, nombre, ambito):
    # Asignar dirección virtual a una variable local
    clave = (ambito, nombre)
    if clave not in self.dir_l:
        self.cont_l += 1
        direccion = self.rango_dir_l + self.cont_l - 1
        self.dir_l[clave] = direccion
    return self.dir_l[clave]

def nueva_var_p(self, nombre, func):
    # Asignar dirección virtual a un parámetro
    clave = (func, nombre)
    if clave not in self.dir_p:
        self.cont_p += 1
        direccion = self.rango_dir_p + self.cont_p - 1
        self.dir_p[clave] = direccion
    return self.dir_p[clave]

# Obtener direcciones
def obtener_dir_cte(self, valor):
    # Obtener direccion de una constante
    return self.dir_cte.get(valor)

def obtener_dir_g(self, nombre):
    # Obtener direccion de una variable global
    return self.dir_g.get(nombre)

def obtener_dir_l(self, nombre, ambito):

```

```

        # Obtener direccion de una variable local
        return self.dir_l.get((ambito, nombre))

def obtener_dir_p(self, nombre, func):
    # Obtener direccion de un parámetro
    return self.dir_p.get((func, nombre))

def obtener_dir_temp(self):
    # Obtener direccion de una variable temporal
    return self.dir_temp.get(self.cont_temp)

def obtener_dir_var(self, nombre, ambito_actual=0,
funcion_actual=None):
    # Obtener direccion de una variable segun su ambito
    # Primero busca en parametros, luego locales, luego globales
    if funcion_actual:
        dir_p = self.obtener_dir_p(nombre, funcion_actual)
        if dir_p is not None:
            return dir_p

    dir_l = self.obtener_dir_l(nombre, ambito_actual)
    if dir_l:
        return dir_l

    dir_g = self.obtener_dir_g(nombre)
    if dir_g:
        return dir_g

    return None

def obtener_func_actual(self):
    # Obtener la funcion actual en la pila de ambitos
    for i in range(len(self.pila_ambitos) -1, -1, -1):
        if 'funcion' in self.pila_ambitos[i]:
            return self.pila_ambitos[i]['funcion']
    return None

```

A diferencia de estos métodos, se definió el método 'nueva_var_temp()' fuera del archivo de la Semántica. Se definió en el archivo del Traductor, ya que este método maneja la creación de variables temporales y estas se crean en el momento de la traducción a cuádruplos. Esta es su implementación:

```

def nueva_var_temp(self):

```

```

        # Asignar dirección virtual a una variable temporal
        self.tabla_vars.cont_temp += 1
        direccion = self.tabla_vars.rango_dir_temp +
self.tabla_vars.cont_temp - 1
        nombre = f"t{self.tabla_vars.cont_temp}"

        # Guardar al diccionario de variables temporales
        self.tabla_vars.dir_temp[nombre] = direccion
        self.gen.operandos_direcciones.push(direccion)
        self.gen.tipos.push('int')
        return nombre, direccion

```

Para llevar a cabo la traducción, se modificó la regla gramatical p_factor:

```

def p_factor(p):
    '''
    factor : ID
            | ID PARENT_A lista_exp PARENT_C
            | CTE_INT
            | CTE_FLOAT
            | NULL
            | PARENT_A exp PARENT_C
            | MENOS factor %prec UMENOS
    '''
    if len(p) == 2:
        if p.slice[1].type == 'ID':
            nombre = p[1]
            print(f"DEBUG: Procesando ID '{nombre}'")
            var_info = tabla_vars.buscar_var(nombre)
            tipo = var_info['tipo'] if var_info else 'int'
            # Obtener direccion virtual
            direccion = tabla_vars.obtener_dir_var(nombre,
tabla_vars.cont_ambitos, None)
            traductor.proces_operando(nombre, tipo, direccion)
            p[0] = (nombre, direccion)
        elif p.slice[1].type == 'CTE_INT':
            direccion = tabla_vars.nueva_cte(p[1])
            traductor.proces_operando(p[1], 'int', direccion)
            p[0] = (p[1], direccion)
        elif p.slice[1].type == 'CTE_FLOAT':
            direccion = tabla_vars.nueva_cte(p[1])
            traductor.proces_operando(p[1], 'float', direccion)
            p[0] = (p[1], direccion)
        elif p.slice[1].type == 'NULL':

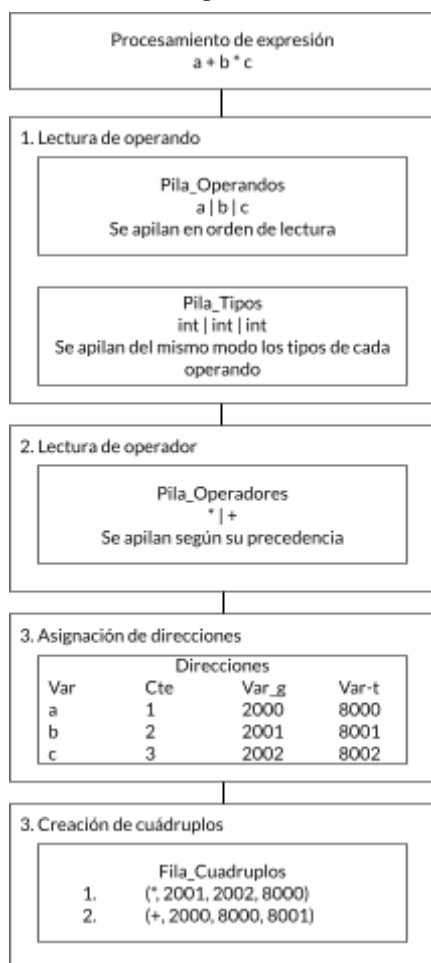
```

```

        traductor.proces_operando(p[1], 'null')
        p[0] = 'null'
    else:
        p[0] = p[1]
    elif len(p) == 4 and p[1] == '(':
        # Expresion entre parentesis
        p[0] = p[2]
    elif len(p) == 3:
        # - factor (UMENOS)
        traductor.proces_umenos()
        p[0] = ('-', p[2])

```

Puntos neurálgicos de la traducción de direcciones



Cuádruplos para estatutos condicionales y ciclos

En el archivo del Traductor se implementó el siguiente método:

```

def nueva_etiqueta(self):
    # Crear una etiqueta
    self.cont_etiquetas += 1

```

```
return f"L{self.cont_etiquetas}"
```

Su propósito es usar una “etiqueta” dónde podrá identificar el momento en el que debe marcar cuando debe hacer un salto hacía otro segmento del código en el caso de que alguna condición sea Falsa.

Los siguientes métodos muestran la traducción de las condicionales if y else. Justo como los métodos anteriormente mencionados, el propósito de estos métodos está comentado justo debajo de su definición:

```
def traduc_if(self, dir_condicion):
    # Traducir if
    label_fin = self.nueva_etiqueta()
    self.gen.cuadрупlos.agregar('if', dir_condicion, None,
label_fin)
    return label_fin

def finalizar_if(self, label_fin):
    # Finalizar if
    self.gen.cuadрупlos.agregar('label', None, None, label_fin)

def traduc_else(self, dir_condicion):
    # Traducir else
    label_else = self.nueva_etiqueta()
    label_fin = self.nueva_etiqueta()
    self.gen.cuadрупlos.agregar('if', dir_condicion, None,
label_else)
    return label_else, label_fin

def salto(self, label_else, label_fin):
    # Manejar saltos
    # Salto al final
    self.gen.cuadрупlos.agregar('goto', None, None, label_fin)
    # Salto a else
    self.gen.cuadрупlos.agregar('label', None, None, label_else)

def finalizar_else(self, label_fin):
    # Finalizar else
    self.gen.cuadрупlos.agregar('label', None, None, label_fin)
```

Los siguientes métodos pertenecen a la traducción del ciclo. El ciclo tiene 2 etiquetas: inicio y fin. El inicio es para volver al inicio del ciclo donde comprobará si la condición se cumple al hacerse una iteración:

```
def traduc_while_inicio(self):
```

```

        # Traducir inicio de while
        label_inicio = self.nueva_etiqueta()
        label_fin = self.nueva_etiqueta()
        self.gen.cuadрупlos.agregar('label', None, None, label_inicio)
        return label_inicio, label_fin

    def traduc_while_condicion(self, dir_condicion, label_inicio,
label_fin):
        # Traducir condición de while
        self.gen.cuadрупlos.agregar('if', dir_condicion, None,
label_fin)

    def finalizar_while(self, label_inicio, label_fin):
        # Finalizar while
        self.gen.cuadрупlos.agregar('goto', None, None, label_inicio)
        self.gen.cuadрупlos.agregar('label', None, None, label_fin)

```

Después de esta implementación, se modificaron las reglas gramaticales `p_condicion` y `p_ciclo` para poder llevar a cabo la traducción:

```

def p_condicion(p):
    '''
        condicion : IF PARENT_A exp_relacion PARENT_C estatuto
                    | IF PARENT_A exp_relacion PARENT_C estatuto ELSE
estatuto
    '''
    if len(p) == 6:
        result, dir_result, tipo_exp = traductor.fin_expresion()
        label_fin = traductor.traduc_if(dir_result)
        traductor.finalizar_if(label_fin)
        print(f" If: condicion {result}")
        p[0] = ('if', result, p[5])
    else:
        result, dir_result, tipo_exp = traductor.fin_expresion()
        label_else, label_fin = traductor.traduc_else(dir_result)
        traductor.salto(label_else, label_fin)
        traductor.finalizar_else(label_fin)
        print(f" If-Else: condicion {result}")
        p[0] = ('if-else', result, p[5], p[7])

def p_ciclo(p):
    '''
        ciclo : WHILE PARENT_A exp_relacion PARENT_C body
    '''

```

```

label_inicio, label_fin = traductor.traduc_while_inicio()
result, dir_result, tipo_exp = traductor.fin_expresion()
traductor.traduc_while_condicion(dir_result, label_inicio,
label_fin)
traductor.finalizar_while(label_inicio, label_fin)
print(f" While: condicion {result}")
p[0] = ('while', result, p[5])

```

Puntos neurálgicos de los cuádruplos if-else y while:



Cuádruplos para funciones

La implementación para la generación de cuádruplos para las funciones es similar a lo anterior hecho: Se usa una etiqueta para identificar que sea una función.

```
def nueva_etiqueta_funcion(self, nombre):  
    # Crear etiqueta para funcion  
    return f"func_{nombre}"
```

Se listan los métodos para la traducción de cuádruplos de las funciones. Para ello, también se deben traducir los parámetros de las funciones si las hay, la llamada de la función y el estatuto de retorno:

```
def iniciar_func(self, nombre):  
    # Iniciar traduccion de funcion  
    label = self.nueva_etiqueta_funcion(nombre)  
    self.gen.cuadрупlos.agregar('label', None, None, label)  
  
    self.funcion_actual = nombre  
  
    def finalizar_func(self):  
        # Finalizar traduccion de funcion  
        self.gen.cuadрупlos.agregar('ENDF', None, None, None)  
        self.funcion_actual = None  
  
    def traduc_param(self, nom_param, dir_param):  
        # Traducir parametro  
        pass  
  
    def traduc_call(self, nom_func, argumentos):  
        # Traducir llamada a funcion  
        for arg_dir in argumentos:  
            self.gen.cuadрупlos.agregar('param', arg_dir, None, None)  
  
        self.gen.cuadрупlos.agregar('call', nom_func, None, None)  
  
    def traduc_return(self, dir_result=None):  
        # Traducir return  
        if dir_result is not None:  
            self.gen.cuadрупlos.agregar('return', dir_result, None,  
None)  
        else:  
            self.gen.cuadрупlos.agregar('return', None, None, None)
```

Y para llevar a cabo la traducción, se modificaron las reglas gramaticales p_func, p_call, p_retorno:

```
def p_func(p):
    '''
    func : VOID ID PARENT_A parametros PARENT_C vars START body END
          | INT ID PARENT_A parametros PARENT_C vars START body END
          | FLOAT ID PARENT_A parametros PARENT_C vars START body END
          | empty
    '''
    if len(p) > 2:
        funcion_actual = p[2]
        tipo_return = p[1]
        parametros = p[4]

        directorio.agregar(funcion_actual, tipo_return, parametros)
        traductor.iniciar_func(funcion_actual)

        tabla_vars.entrada_ambito()
        for nom_param, tipo_param in parametros:
            dir_param = tabla_vars.nueva_var_p(nom_param,
funcion_actual)
            tabla_vars.declar_var(nom_param, tipo_param, p.lineno(2))
            tabla_vars.pila_ambitos[-1][nom_param]['direccion'] =
dir_param

        print(f" Funcion: {p[2]} tipo {p[1]}")
        traductor.finalizar_func()
        tabla_vars.salida_ambito()
        funcion_actual = None
    p[0] = None

def p_call(p):
    '''
    call : ID PARENT_A lista_exp PARENT_C PUNTO_COMA
    '''
    nombre_func = p[1]
    args = p[3] if p[3] else []

    dirs_agrs = []
    for arg in args:
        if isinstance(arg, tuple) and len(arg) == 2:
            dirs_agrs.append(arg[1]) # valor y direccion
```

```

        else:
            dirs_agrs.append(arg)

    traductor.traduc_call(nombre_func, dirs_agrs)
    print(f" Call: {p[1]} ({p[3]})")
    p[0] = ('call', p[1], p[3])

def p_retorno(p):
    '''
    retorno : RETURN exp PUNTO_COMA
    '''
    print("DEBUG: Procesando return...")
    result, dir_result, tipo_exp = traductor.fin_expresion()
    print(f"DEBUG: result={result}, dir_result={dir_result},
tipo={tipo_exp}")
    traductor.traduc_return(dir_result)
    print(f" Return: {result}")
    p[0] = ('return', result)

```

Puntos neurálgicos de los cuádruplos de funciones:

